3rd Year 2nd Semester

University of Dhaka
Department of Computer Science and Engineering

Session: 2022–2023

CSE 3216: Software Design Patterns Lab

Assignment Topic

Singleton Pattern Implementations in Python and Their Practical Significance

Submission Date:

21/04/2026

Submitted By:

H.M. Mehedi Hasan
Roll: 13

Introduction

This report evaluates six Singleton Pattern implementations using a MongoDB connection manager. The experimental analysis focuses on three aspects:

- instance validation,
- thread safety under concurrent execution,
- performance in terms of initialization and repeated access.

The implementations evaluated are:

- Eager Initialization
- Lazy Initialization (basic)
- Synchronized Method
- Double-Checked Locking
- Bill Pugh Implementation
- Enum-based Singleton

1. Instance Validation Results

Eager Initialization

Object IDs

`2610221224704, 2610221224704, 2610221224704, 2610221224704, 2610221224704`

Interpretation

All repeated calls returned the same object ID, so only one instance was created.

Lazy Initialization (basic)

Object IDs

`2610224249824, 2610224249824, 2610224249824, 2610224249824, 2610224249824`

Interpretation

Sequential calls returned the same object ID, indicating correct singleton behavior in non-concurrent execution.

Synchronized Method

Object IDs

`2610224245168, 2610224245168, 2610224245168, 2610224245168, 2610224245168`

Interpretation

A single instance was maintained across repeated sequential calls.

Double-Checked Locking

Object IDs

`2610224443168, 2610224443168, 2610224443168, 2610224443168, 2610224443168`

Interpretation

All repeated calls produced the same object ID, confirming singleton behavior.

Bill Pugh Implementation

Object IDs

```
2610224570208, 2610224570208, 2610224570208, 2610224570208, 2610224570208
```

Interpretation

Only one object instance appeared during sequential execution.

Enum-based Singleton

Object IDs

```
2610224241760, 2610224241760, 2610224241760, 2610224241760, 2610224241760
```

Interpretation

All repeated accesses returned the same object ID.

Table 1. Instance Validation Summary

Implementation	Unique Objects	Single Instance Maintained
Eager Initialization	1	Yes
Lazy Initialization (basic)	1	Yes
Synchronized Method	1	Yes
Double-Checked Locking	1	Yes
Bill Pugh Implementation	1	Yes
Enum-based Singleton	1	Yes

The sequential validation results show that all six implementations preserve singleton behavior when accessed without concurrency.

2. Thread Safety Analysis

Eager Initialization

Thread IDs Output

```
2610223982640, 2610223982640, 2610223982640, 2610223982640, 2610223982640,
2610223982640, 2610223982640, 2610223982640, 2610223982640, 2610223982640,
2610223982640, 2610223982640, 2610223982640, 2610223982640, 2610223982640,
2610223982640, 2610223982640, 2610223982640, 2610223982640, 2610223982640
```

Interpretation

All threads returned the same object ID. The implementation remained thread-safe in the experiment.

Lazy Initialization (basic)

Thread IDs Output

```
2610224564688, 2610224564544, 2610224563440, 2610224564880, 2610224564352,
2610224564592, 2610224564736, 2610224564976, 2610224564928, 2610224565024,
2610224564640, 2610224564448, 2610224564784, 2610224564496, 2610224565168,
2610224564160, 2610224564400, 2610224564832, 2610224565120, 2610224565072
```

Interpretation

Different threads produced different object IDs. The implementation failed under concurrency and created multiple instances.

Synchronized Method

Thread IDs Output

```
2610223984128, 2610223984128, 2610223984128, 2610223984128, 2610223984128,
2610223984128, 2610223984128, 2610223984128, 2610223984128, 2610223984128,
2610223984128, 2610223984128, 2610223984128, 2610223984128, 2610223984128,
2610223984128, 2610223984128, 2610223984128, 2610223984128, 2610223984128
```

Interpretation

All threads received the same object ID, indicating correct thread-safe behavior.

Double-Checked Locking

Thread IDs Output

```
2610234317200, 2610234317200, 2610234317200, 2610234317200, 2610234317200,
2610234317200, 2610234317200, 2610234317200, 2610234317200, 2610234317200,
2610234317200, 2610234317200, 2610234317200, 2610234317200, 2610234317200,
2610234317200, 2610234317200, 2610234317200, 2610234317200, 2610234317200
```

Interpretation

All threads returned the same object ID, so the implementation remained consistent under concurrency.

Bill Pugh Implementation

Thread IDs Output

```
2610234316720, 2610234316720, 2610234316720, 2610234316720, 2610234316720,
2610234316720, 2610234316720, 2610234316720, 2610234316720, 2610234316720,
2610234316720, 2610234316720, 2610234316720, 2610234316720, 2610234316720,
2610234316720, 2610234316720, 2610234316720, 2610234316720, 2610234316720
```

Interpretation

The same object ID was preserved across all threads.

Enum-based Singleton

Thread IDs Output

```
2610224241760, 2610224241760, 2610224241760, 2610224241760, 2610224241760,
2610224241760, 2610224241760, 2610224241760, 2610224241760, 2610224241760,
2610224241760, 2610224241760, 2610224241760, 2610224241760, 2610224241760,
2610224241760, 2610224241760, 2610224241760, 2610224241760, 2610224241760
```

Interpretation

The same object ID appeared in all thread calls, confirming thread-safe behavior in practice.

Table 2. Thread Safety Summary

Implementation	Thread-Safe	Unique Objects in Threads
Eager Initialization	Yes	1
Lazy Initialization (basic)	No	20
Synchronized Method	Yes	1
Double-Checked Locking	Yes	1
Bill Pugh Implementation	Yes	1
Enum-based Singleton	Yes	1

The multithreading results clearly distinguish the unsynchronized lazy approach from the remaining implementations. Lazy Initialization is the only implementation that produces multiple objects under concurrent access.

3. Performance Results

Table 3. Single-Run Performance Summary

Implementation	Avg Init Time (s)	Avg Access Time (s)
Eager Initialization	0.1846	0.00000157
Lazy Initialization (basic)	0.1708	0.00000241
Synchronized Method	0.1970	0.00000361
Double-Checked Locking	0.2037	0.00000257
Bill Pugh Implementation	0.1928	0.00000414
Enum-based Singleton	0.1780	0.00000052

Table 4. Multi-Run Average Performance

Implementation	Mean Init Time (s)	Mean Access Time (s)
Eager Initialization	0.213170	0.000002369
Lazy Initialization (basic)	0.178316	0.000002181
Synchronized Method	0.183948	0.000003836
Double-Checked Locking	0.184820	0.000002437
Bill Pugh Implementation	0.185296	0.000003963
Enum-based Singleton	0.182444	0.000001026

Performance Interpretation

The selected run shows that Lazy Initialization has the lowest initialization time, while Enum-based Singleton has the lowest repeated access time. The multi-run average provides a more stable comparison: Enum-based Singleton remains the most efficient for repeated access, while Lazy Initialization keeps relatively low initialization cost but fails under concurrency. Synchronized Method and Bill Pugh Implementation show comparatively higher access overhead.

4. Comparative Summary

Overall Findings

- All six implementations maintain a single instance in sequential execution.
- Lazy Initialization (basic) fails under concurrent execution and produces multiple instances.
- Eager Initialization, Synchronized Method, Double-Checked Locking, Bill Pugh Implementation, and Enum-based Singleton remain thread-safe.
- Enum-based Singleton provides the best repeated access performance in both the selected run and the multi-run average.
- Synchronized Method and Bill Pugh Implementation show higher access overhead.
- Double-Checked Locking remains thread-safe and provides competitive access performance.

5. Conclusion

Final Conclusion

The experimental results show that all implementations satisfy singleton behavior in sequential execution. However, only thread-safe implementations maintain correctness under concurrent execution. Lazy Initialization (basic) is the only implementation that consistently fails in thread-based access. Based on the observed performance values, Enum-based Singleton provides the best repeated access efficiency, while Double-Checked Locking remains a balanced thread-safe solution.